The Project in Plain English

Oct 19, 2026 · @barnn

The idea in one minute

We are building a tool that tells a person which hours of a building's electricity use deserve a second look. It does not say that something is wrong. It says that the use looks unusual for that building at that time.

The midterm project asks one question: when our tool marks an hour as unusual, does that match what people independently marked as unusual? We also ask which choices in building the tool make the match better.

Why it matters. Universities and other large buildings use a lot of electricity, and nobody can watch every meter every hour. Equipment can be left running at night, a fault can raise the use for days, or a sensor can fail. A tool that points a person to the few hours worth checking saves time. But only if the pointers are right, and that is exactly what we test.

How the tool works

Think of a caretaker who has looked after a building for years. The caretaker knows its routine. Offices fill up on weekday mornings and empty in the evening. Hot days mean more cooling. Weekends are quiet. If the lights and machines are running at full load at three in the morning, the caretaker notices, even without a single alarm going off.

Our tool does the same in three steps.

1. **Learn the routine.** For every building and every hour, the tool works out how much electricity we would normally expect. It looks at the time of day, the day of the week, the weather and facts about the building, such as its size and its main use.
2. **Compare with reality.** It measures how far the real reading is from the expected one. A small gap is normal. A big gap is unusual.
3. **Mark the unusual hours.** Hours with a big gap, compared with that building's own normal gaps, are marked. The result is a ranked list, with the most unusual hours first, for a person to look at.

Two details matter. Each building is judged against its own habits, because a hospital and a library have very different routines. And the line between "normal" and "unusual" is drawn using older data and saved before we look at the newest data, so we cannot quietly adjust it to flatter the results.

The information we give the tool

In data science these clues are called features. Here they are in plain words, in four groups.

Time clues. The hour of the day, the day of the week, and whether it is a weekend. These capture the building's routine. We removed the month on purpose. The test months never appear in the training months, so a model that leans on the month would be guessing about something it has never seen.

Weather clues. Air temperature, dew point (how humid the air feels), cloud cover, rain in the last hour, sea-level pressure and wind speed. Weather explains why cooling or heating rises on some days. When a weather reading is missing we carry the last known value forward in time, never backward, because using a later reading to fill an earlier gap would be a form of peeking.

Building facts. The building itself (each building has its own number), the site it belongs to, the floor area, the main use (for example education or office), the year it was built and the number of floors. The last two are often blank in the data, and the models accept blanks. These let the tool learn that a large laboratory and a small office behave differently.

History clues. The reading from the previous hour, the reading from the same hour yesterday and the reading from the same hour last week. They are the most powerful clues. They are looked up by clock time, not by row position in the table, because a few rows are missing and a row-based lookup would then fetch the wrong hour. Readings are put on a log scale, which means the tool pays attention to relative change (a jump of a fifth) instead of raw size, so big and small buildings are treated fairly.

What we never feed to the tool. The human-made labels. They are used only afterwards, to score the marks.

What we have not tried yet. Readings from two or three hours back, a rolling average over the last day, holidays, and what neighbouring buildings are doing. These are listed under remaining work.

Models A, B and C, and what we did with them

What kind of model it is. All three are the same kind of model, called gradient boosting. Picture a team of very small decision makers. The first one makes a rough guess of expected use. The second one studies where the first was wrong and corrects it. The third corrects what is left, and so on for hundreds of rounds. Added together, they make a good prediction. This type of model works well on tables of mixed clues such as ours and trains quickly.

How we built them. We trained each one on the earliest part of the year, up to the middle of September. We tuned the settings on the next period, up to early November, and chose one set of settings. Then we froze everything and tested on the last period, November and December, which the models had never seen. All three models use exactly the same settings and the same training rows. The only difference is which clues each model is allowed to see.

- **Model A** sees time clues, weather and building facts only. It knows nothing about what the building used recently.
- **Model B** sees everything in A, plus the same hour yesterday and the same hour last week.
- **Model C** sees everything in B, plus the previous hour.

This design is a controlled experiment. If C finds more events than B, the extra finding comes from the previous hour and nothing else.

The other three methods we compared. Same as last hour simply says that this hour should be like the one before. Typical value says that this hour should be like the usual value for that building at that hour on that kind of day. Fixed percentile does not predict anything. It marks any hour that is higher than almost all the building's own earlier hours. These three act as reality checks: a clever model should beat them, or it is not earning its place.

What we did with each one, in order.

1. Trained the model and asked it for the expected use in every hour of the test period.
2. Measured the gap between real and expected use.
3. Compared each gap with that building's own normal gap, to get a score. A large score means an unusual hour.
4. Set the score cut-off using the validation period only, and saved it to a file before opening the test period.
5. Marked the test hours above the cut-off.
6. Compared the marks with the human-labelled events.
7. Repeated the comparison with every method allowed the same number of marks, to be fair.

What happened, in words. Model A and Model B found about the same share of events, even though B predicts better. Model C found clearly more, and it kept flagging long events better than the simple copy-the-last-hour rule. The simple rule and Model C were close overall. So the key ingredient was the previous hour, not the extra history, and not the complexity of the model.

How we move through the work

The whole project lives in one notebook, a single file that runs from the raw data to the final tables. It is organised in five parts plus a set of data checks.

1. **Data checks.** Look for duplicates, negative values, gaps and odd timing.
2. **Part A, labels.** Join the human labels to the readings and group them into events by length.
3. **Part B, predictors.** Build the clues, train models A, B and C and the simple methods, and measure how well each predicts.
4. **Part C, cut-offs and flags.** Turn gaps into scores, set and save the cut-offs, and mark the test hours.
5. **Part D, evaluation.** Score the marks against the events at the saved cut-offs.
6. **Part E, equal comparison.** Compare all methods when each may mark the same share of hours.

Each part saves its results as files, so a single part can be repeated without redoing the others. The settings (paths, dates and rules) sit in one cell at the top, so anyone on the team can change them in one place.

Questions the professor may ask, with short answers

Why not just use the best predictor? Predicting well and detecting well are different jobs. A tool that copies the last hour predicts almost perfectly, yet it can miss an event that lasts many hours, because the event quickly becomes the new normal. We measure both jobs separately.

Does better prediction make the tool overlook persistent anomalies? Partly yes. For the copy-the-last-hour rule the concern holds. For the full model it is smaller. We report this as a partial answer, not a final one.

Are the labels the truth? No. People made them by looking at plots, and they come from the same readings we use, so they are not independent. We say plainly that we measure agreement with these labels, not proof of waste or faults.

Did you peek at the test data? The cut-offs were saved before the test period was scored. After seeing the test results we noticed that the marks per method differed from our target, and we kept the cut-offs unchanged on purpose. We then added an equal-rate comparison so the methods are judged fairly.

Why compare at equal alarm rates? A method that marks more hours will find more events for that reason alone. Giving every method the same number of marks removes that advantage.

Why a time split and not a random one? A random split would let the model learn from hours right next to the ones it is tested on. A time split mimics real use, where we only know the past.

Why remove the month? The test months never appear in training, so the month cannot help and could mislead.

Why are models A, B and C built the same way? So that the only difference is the clues. That lets us say which clue matters.

Why did you not use a deep neural network? The goal is to learn which design choices matter, not to chase the strongest model. A simpler model is faster to train and easier to explain. A deep model is a possible later comparison.

What does a flag mean? Use that is unusual for its context. It is a candidate for human review. It does not prove waste, a fault or a saving.

What is still untested? Why one shared threshold does worse than per-building thresholds is still only a guess. The idea that model C keeps flagging long events because it looks at yesterday and last week was tested on 2 November, and the answer is in the persistence test below. We say so in the report.

What is left to do? More history clues, a check with other detectors, a look at which building types are hard, and the final written report and ranked review list.

How we check whether it is right

The first version of our project could mark hours as unusual, but it had no way to say whether the marks were correct. Counting how many hours were marked tells you nothing about accuracy. We needed an answer key.

The answer key is a public data set in which people looked at graphs of building electricity use and marked the periods that looked unusual. We call these periods **events**. Some events last a single hour. Others last for days. We keep the answer key hidden from the tool. It is used only afterwards, to score the tool's marks.

We then ask simple questions. Of all the events people marked, how many did our tool also mark at least once? Of the places our tool marked, how many matched an event? The first question is about finding things. The second is about being trustworthy. A tool that marks everything finds every event but is useless, so we always look at both.

The answer key has limits, which we state openly. It was made by people judging graphs, so it shows agreement with human judgment, not absolute truth. It also comes from the same family of data as our electricity readings, so it is not fully independent. Hours that people did not mark may still be unusual, which means our tool can look worse than it really is.

What we have done so far

1. We fixed the foundations. Our first version had three hidden mistakes, and the professor's feedback pointed at them. The first mistake was information from the future leaking into the training. We now split the data by date, so the tool learns only from the past and is tested on a later period it has never seen. The second was in how we looked back in time. The tool uses earlier readings, such as the previous hour, and our first way of fetching them was slightly off wherever hours were missing. We now give every building a complete hourly timeline and fetch history by time, not by row. The third was a single "unusual" line used for every building. Each building now has its own.

2. We cleaned the data. Long stretches of zero readings usually mean a broken meter, not a quiet building, so we treat them as missing. One group of buildings reported energy in a different unit, so we converted it. We checked for duplicates, negative values and gaps.

3. We added the answer key. We joined the human-made labels to the readings and organised them into events, so the tool's marks can be scored event by event.

4. We built six ways to mark unusual hours and compared them. Two are simple: copy the previous hour, or use the usual value for that building, hour and type of day. Three use one learning model that predicts expected use, given different amounts of history. The first sees only calendar, weather and building facts. The second also sees the same hour yesterday and last week. The third also sees the previous hour. The sixth does not predict at all. It marks any hour above the building's own high-use level. Because the model is identical in the three versions, any difference comes from the information it was given.

5. We made the comparison fair. Our first cut-offs marked very different amounts of hours in the test period, which made some methods look better or worse for the wrong reason. We kept those results, said so openly, and added a second comparison where every method may mark the same share of hours.

6. We put everything in one notebook. It runs from the raw files to the final tables in one pass, and we have run it on the real data. Anyone on the team can repeat it.

7. We wrote it up. The progress report answers each of the professor's comments, and the slides tell the same story in a few minutes.

What we found

The hour just before is the most useful clue. Electricity use changes slowly. When the previous reading is known, a sudden jump stands out clearly. Without it, the tool compares each hour with a typical day and misses many events. This was the biggest single difference we saw.

A simple rule does almost as well as the learning model. Predicting that this hour equals the last one found about as many events as the full model. That is a useful reality check. A complicated tool must beat a simple one to earn its place.

Predicting well is not the same as detecting well. A model that gets closer to the real numbers did not necessarily find more events. Looking further back, to yesterday and last week, made predictions better but did not help find events.

The professor's question about long-lasting problems has a mixed answer. If a fault keeps use high for many hours, a rule that only copies the previous hour soon treats the high level as normal and stops flagging. The fuller model keeps flagging such events more often. So the worry is real for the simple rule and smaller for the model. We later tested it directly (see the section on what we did since the midterm).

How the line is drawn matters. When every building shares one line, the methods without the previous hour did much worse than when each building had its own line.

Most marks do not match a labelled event. Even the best setup misses a large part of the events and also marks many hours that people did not label. The honest summary is modest: the tool is a helpful way to rank hours for a person to check, not a finished detector.

What we did since the midterm (2 November 2026)

We checked everything from the very beginning. We ran all the notebooks again, in order, in a clean folder. Everything finished without errors, and a second clean run gave exactly the same numbers as the first. Compared with the midterm, the parts that depend only on the data match exactly. The parts that depend on the three trained models are slightly different (a model trained again is never perfectly identical), but the methods come out in the same order.

We tried a completely different kind of detector. It is called an Isolation Forest. Instead of predicting what a building should use, it looks for hours that are easy to separate from all the others, a bit like spotting the odd one out in a crowd. We built two versions. Both found fewer labeled events than the simple methods: about 29 out of 100 events at the same alarm budget, against 35 for Model C and 39 for the plain copy of the previous hour.

We added margins of error. Counting events found is not enough, because with only 587 test events a few points of difference could be luck. We repeated the count many times, each time picking buildings at random, to see how much the answer moves. The result: Model C and the plain copy of the previous hour are tied (their gap, about 3 events in 100, is smaller than the margin of error). Model C is clearly better than the typical-value baseline and than Models A and B. The Isolation Forest is clearly worse than the plain copy of the previous hour.

We looked inside the events. Among events that a method does find, it flags about half to three quarters of the hours inside the event, and the first flag usually comes within one hour of the start.

We tested the worry about labeled hours in training. Until now the labeled unusual hours were hidden when we set the cut-offs but were still present when the models learned. We removed them from learning too. It did not hurt: Model C found 38 events in 100 instead of 35, which brings it level with the plain copy of the previous hour. This is one run, so we will say "similar or slightly better" and not more.

We tested giving each building its own cut-off. It finds more events, but it flags far too many hours on the test period (up to 7.5% of hours instead of 0.5%). More events found by flagging more hours is not a better method, so we keep the single shared cut-off.

We answered the professor's question about long-lasting anomalies. We split the events by how long they last and checked how many each method finds. The plain copy of the previous hour does very well on one-hour spikes (about 41 in 100) but poorly on events longer than 12 hours (about 33 in 100), because after the first hour the previous hour already contains the problem. Model C, which also looks at yesterday, last week and the weather, finds about 50 in 100 of the long events, which is clearly more than the plain copy. So using history does not make our best model overlook long anomalies, but copying the last hour does. One more thing we saw: inside a long event, every method flags fewer hours as the event goes on (Model C: 13.5% of hours at the start, 4.6% after 12 hours). We do not know why, and we will say so.

We tested long-lasting problems directly, and looked at the marks that matched nothing. At 1% of hours marked, for each building, Model C finds about 50 in 100 of the events longer than 12 hours, against about 33 in 100 for the plain copy of the previous hour. That gap (about 17 points) is bigger than the margin of error. On one-hour events it is the other way round: the plain copy finds about 41 in 100 and Model C about 26. So the worry is real for the simple rule and not for Model C. For the 30 marks that matched no label (a sample), Claude looked at the graphs and sorted them: 18 looked really unusual, 4 plausible, 6 false alarms and 2 could not be told. This is Claude's own judgement, not proof, and a teammate should check it. It suggests that part of the low precision comes from stretches that nobody labeled. We also retrained the models without the labeled hours; see the project folder results/label_excluded/. Everything now runs from one list of packages (requirements.txt).

We tested on a third data set, with problems we planted ourselves (9 November). CU-BEMS is one office building in Bangkok, measured every minute, with no labels from people. We turned it into hourly numbers for each of its seven floors, then hid made-up problems in the last three months: use multiplied by 1.3, 1.6 or 2, or dropped almost to zero, lasting 1, 3, 8 or 24 hours. Because we know exactly where they are, we can count how

many each method finds. We repeated this 20 times with the problems in different places. When the tool marks 2% of hours, Model C finds about 41 in 100 problems, the plain copy of the previous hour about 32, and the Isolation Forest about 33. When it marks only 1% of hours the gap is small and not clear (27, 25 and 23). A drop to zero is easy for every method; a rise of only 30% is hard for all (at most 19 in 100). For problems that last 24 hours, the plain copy of the previous hour flags the first hour but then almost stops (3.5% of hours at the start, about 1% later), while Model C keeps flagging (about 6% at the start, about 4% later). That gives the professor's question the same answer as the real labels: copying the last hour gets used to a long problem, Model C much less. Important limits: it is one building, the problems are simple and made by us, so this is not proof that the tool works in real life. One of the files (floor 6, 2019) stops on 22 October, which we still have to check.

What this means in plain words. The best simple predictor is the one that looks at the previous hour, and the plain copy of the previous hour is just as good at finding events. So the project's honest message is: history about the building matters a lot, a fancy model adds precision in the top of the review list but not more events found, and detectors that ignore history (Isolation Forest, a fixed limit) do worse. We will not claim that our system beats every method.

What is left, from 2 to 17 November

1. Confirm with each other that the test period is used only once with frozen settings.
2. Answer the professor's question about long-lasting anomalies (done on 2 November: see above).
3. Cross-check the ratings of the 30 flags that match no label (done on 2 November, but Claude did the rating, so a teammate should check it).
4. Third dataset, CU-BEMS (a Bangkok office building with no labels): done on 9 November. Still to do: check the short floor 6 file.
5. Build the simple filter that marks a flag as explained when most buildings on a site go odd at the same time.
6. Produce the ranked review list a person would read.
7. Write the report in the AIT template (introduction, problem, related works, datasets, method, results, discussion on performance, interpretability and complexity, conclusions), make the slides, tidy the folder.

The final deadline is 17 November 2026.

What is left, and what the finished project looks like

The midterm shows that the idea works in part and tells us where to improve. The final project will act on what we learned.

Improve how the line is drawn (done on 2 November: we kept the shared line). Today the line between normal and unusual is set from normal validation hours only, not from the labeled events (labeled hours are left out when it is set). We will let the labels from an earlier period guide the choice for each building, then freeze it and score it once on the later period. This should find more events and should also fix the uneven numbers of marks we saw.

Try a different kind of detector (done on 2 November). So far we compare how far each hour is from a prediction. We will add a method that looks directly for hours that are unlike all the others, and compare it with the others on equal terms.

Look inside events (done on 2 November). At the moment an event counts as found if one hour is marked. We will measure how many of its hours are marked and how late the first mark comes. For a real problem, early warning matters.

Remove marks that have an ordinary explanation. Holidays, extreme weather and what neighbouring buildings are doing can explain a jump. A filter for these should reduce false alarms without losing real events.

Make the evidence stronger (margins of error done on 2 November; the second data set, CU-BEMS, done on 9 November). We will give honest margins of error that treat each building as one unit, test more history clues, and run a second check on a different building data set where we insert problems at known times, so the right answer is certain.

The finished project will contain:

- one notebook that runs from raw data to final results,
- a final report and presentation that answer the research question with evidence,
- a shared code repository with clear instructions so that anyone can repeat the work,
- a ranked list of unusual hours for each building, with the clues that explain why each one was marked.

The tool will still not decide what is wrong. It will tell a person where to look first, and we will show how much that guidance can be trusted.

What the project does not claim, and common questions

We do not claim that a marked hour is waste, a fault or a saving. Public data has no information about who was in a building, what events were held or when repairs happened, so we cannot name causes. We claim only that the hour was unusual for that

building at that time.

Does the tool find problems? It finds some of the periods that people marked as unusual, and it finds them best when it knows the previous reading. It also marks many hours that nobody labelled.

Why not just use a simple rule? A simple rule does well overall, and that is one of our findings. The fuller model is better on long-lasting events, which is the case the professor asked about.

Can we trust the answer key? It is a good check against human judgment, but it is not perfect and it is not fully independent of our data. We say so in the report.

Why did you change the comparison after seeing the results? Our first cut-offs marked very different numbers of hours, which made the comparison unfair. We kept the original results, reported them, and added a fair comparison. We say plainly that this came after we looked at the test results.

What would make this useful in real life? A building manager could receive a short daily list of the most unusual hours, check them, and report back. Those answers would then become better labels for the next version.

Where we honestly stand, and the work left to finish

This section is for us, so that we are not caught off guard. Read it before the presentation.

How far along are we? Roughly half to three fifths of the whole project. The hard and risky part is done: clean data, a fair test that does not leak the future, and the comparison methods. What remains is the second half of the research question, which is how the choice of detector and the choice of cut-off change the match with the labels.

The weak spots a professor may press on.

1. **The simple rule nearly ties the full model.** This is a real finding, not a failure. Say it first: it shows why we compared against a simple baseline. The model's advantage is on long-lasting events.
2. **Only one of the three design choices is tested well.** Clues are tested well. The cut-off is partly tested. The detector is barely tested, because so far we compare predictors, not different kinds of detectors. The Isolation Forest comparison is the planned fix.
3. **Most marks do not match a labelled event.** We report it. We have not yet looked at why. Some of those marks may be real unusual hours that the labelers missed.
4. **The answer key is not fully independent.** People made it from plots of the same data. Every claim is about agreement with those labels.

5. **One year, one split, no margins of error that treat a building as one unit.** Small differences between close methods could be noise.
6. **The ranked list for a person to review does not exist yet.** It is the end product of the project.
7. **We looked at the test results before noticing the uneven mark rates.** We kept the cut-offs frozen and said so. Be ready to explain that calmly.

The work left, in order of importance.

1. **Compare a different kind of detector** with the existing methods on equal terms. About one to two days.
2. **Test the professor's question directly:** does the tool overlook long-lasting problems, and does a scoring rule that notices a persistent gap help? About one day.
3. **Choose the cut-off per building using labels from the earlier period,** freeze it, then score once on the later period. About one day.
4. **Study the marks that do not match a label,** with a few worked examples. About one day.
5. **Add more history clues** such as two and three hours back and a daily average. About half a day.
6. **Make the evidence stronger:** margins of error that treat each building as one unit, and a check on a second split or a second data set. About one day.
7. **Build the ranked review list** with a short note on how to use it. About one day.
8. **Write the final report and presentation.** About two days.

That is about six to eight working days of effort in total, or three to four days each when two people share it. This is our estimate. Check it against the real final deadline.

If time runs short, drop the second data set and the deep model first. Keep the detector comparison and the ranked review list, because they answer the research question and deliver the product.

What to do tonight. No new experiments. Check the report numbers against the result files, let Arpan read the report and the script once, export both PDFs, open them and look at every figure and table, and submit before the deadline.